

Artificial Intelligence

[Notes by Piai Luca]

Academic Year 2024–2025

Additional well-written notes on the same topic are available at the following link:
https://github.com/lovomanuel/Unipd-Notes/blob/main/Artificial_Intelligence/Artificial_Intelligence.pdf. These notes were also used as a reference in the preparation of this document.

Contents

1	Intelligent Agents	1
1.1	Agents and environments	1
1.2	Task environment	2
1.3	Agent functions and programs	4
1.4	Agent types	5
1.5	How the components of agent programs work	5
2	Stable matching problem	7
2.1	Matching under preferences	7
2.2	Stable matching problems	7
2.2.1	Stable Marriage (ideal) formulation	8
2.3	Gale Shapley algorithm	9
2.3.1	Features of the algorithm	10
2.4	Formulation extensions	10
2.4.1	Ties in preferences	10
2.4.2	Incomplete preferences	11
2.4.3	Ties and incomplete preferences	12
2.5	Manipulation and strategies	12
3	Solving problem by searching	15
3.1	Problem solving agent	15
3.2	Problem formulation	16
3.3	The Search Tree	16
3.3.1	Search Tree algorithm infrastructure	17
3.4	Search strategies	18
3.4.1	Uninformed strategies	19
3.4.2	Informed strategies	19
3.4.3	Heuristics functions	21
3.5	Local search algorithms	22

3.5.1	Hill-climbing search	23
3.5.2	Simulated annealing search	24
3.5.3	Local beam search	25
3.5.4	Genetic algorithms	25
4	Constraint Satisfaction Problems	27
4.1	Basic definitions	27
4.1.1	Varieties of CSPs	28
4.2	Search formulations	29
4.2.1	Standard search formulation	29
4.2.2	Backtracking search	30
4.2.3	Backjumping	32
4.2.4	No-good	32
4.2.5	Forward checking	33
4.3	Constraint propagation	33
4.3.1	Node consistency	34
4.3.2	Arc consistency (AC)	34
4.3.3	Path consistency (PC)	35
4.4	Local search for CSPs	36
4.4.1	Min-Conflict Local Search	37
4.4.2	Deal with local minima	37
4.4.3	Constraints weighting	38
4.5	Structure of the problem	38
4.5.1	Tree-structured CSP	39
4.5.2	Almost tree-structured	40
4.5.3	Tree decomposition	40
4.6	Soft CSP	42
4.6.1	Fuzzy CSP	43
4.6.2	Weighted CSP	44
4.6.3	Other soft CSPs	44
4.6.4	Multi-criteria problems	44
4.6.5	Fundamental operations	45
4.6.6	How to find optimal solutions	46
5	Multi-agent decision making	49
5.1	Defining preferences	49
5.2	Preferences in multi-agent systems	50

5.3	Voting theory	50
5.4	Multi-agent AI scenarios	52
5.5	Formalisms to model compactly preferences	52
5.5.1	Soft constraints	53
5.5.2	CP-net	53
5.6	Sequential voting	54
6	Bayesian Networks	57
6.1	Introduction	57
6.2	Syntax of Bayesian Networks	57
6.2.1	Example: The Burglary Alarm Network	57
6.2.2	Conditional Probability Tables	59
6.3	Compactness of Representation	59
6.3.1	Comparison with Full Joint Distribution	59
6.4	Semantics of Bayesian Networks	59
6.4.1	Example Calculation	59
6.5	Summary	60

Chapter 1

Intelligent Agents

1.1 Agents and environments

There exists many view of AI, they fall into four categories: *thinking humanly, thinking rationally, acting humanly, acting rationally*. In our course we will consider as AI, acting rationally. In this course we study the problem of building intelligent agents.

Definition 1 (Intelligent agent). *Ideally, an intelligent agent takes the best possible action in a situation. That is, an agent is intelligent if it "does the right thing".*

Definition 2 (Agent). *An agent is an entity that perceives its environment through sensors and acts upon that environment through actuators.*

Definition 3 (Rational agent - informal). *A rational agent is an agent that acts so as to achieve:*

- *the best outcome or*
- *the best expected outcome when there is uncertainty.*

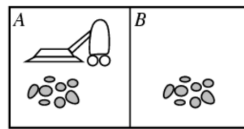
Mathematical point of view of the agent.

Definition 4 (Agent function). *The agent function maps percept histories to actions:*

$$f : P^* \rightarrow A.$$

An agent is completely specified by the agent function mapping percept sequences to actions.

Definition 5 (Agent program). *The agent program runs on the physical architecture to produce the agent function.*



- **Percepts:** location and contents, e.g., [A, Dirty]
- **Actions:** Left, Right, Suck, NoOp
- **A simple agent function**
 - if the current square is dirty, then suck;
 - otherwise, move to the other square

Figure 1.1: Vacuum Cleaner example.

Percept Sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Dirty], [A, Clean]	Right
[A, Clean], [B, Dirty]	Suck
[A, Clean], [B, Clean]	No-op
...	...

Figure 1.2: Vacuum Cleaner: partial tabulation of the agent function.

Many different vacuum cleaner agents can be defined by changing the right-hand column in various ways. But what is the right way to fill out the table? Since we're looking for a rational agent, we need to define the **performance measure**: an objective criterion for success of an agent's behaviour. For instance, for the vacuum cleaner, we can define the performance measure as the amount of dirt cleaned up, or the amount of time taken.

Therefore, the rational agent is the one that, for each possible percept sequence, it selects an action that is expected to maximize its performance measure. The decisions are taken based on the evidence provided by the percept sequence and by whatever built-in knowledge the agent has.

1.2 Task environment

To design a rational agent we must first specify its task environment.

Definition 6 (Task environment). *A task environment is specified by the PEAS description (Performance measure, Environment, Actuators, Sensors).*

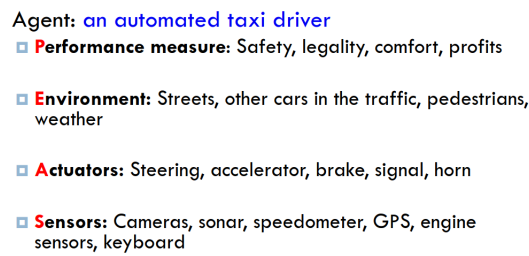


Figure 1.3: Example of Task environment.

Fully observable and partially observable task environment

- **Fully observable:** agent's sensors give it access to the complete state of the environment at each point in time.
- **Partially observable:** due to the presence of noise in the sensors.
- **Unobservable:** when the agent has no sensors.

Deterministic or stochastic task environment

- **Deterministic:** the next state of the environment is completely determined by the current state and action executed by the agent.
- **Stochastic:** there is uncertainty on the next state and it is expressed with probabilities.
- **Non-deterministic:** there is uncertainty on the next state but no probabilities are available.
- **Uncertain:** not fully observable and non deterministic.

Episodic or sequential task environment

- **Episodic:** the agent's experience is divided into atomic episodes. Each episode consists of the agent perceiving and performing a single action. The choice of action in each episode depends only on the episode itself.
- **Sequential:** a current decision may affect future decision.

Static or dynamic task environment

- **Static:** the environment is unchanged while an agent is deliberating.
- **Dynamic:** continuously asking the agent what it wants to do.
- **Semi-dynamic:** if the environment itself does not change with the passage of time but the agent's performance score does.

Discrete or continuous task environment

- **Discrete:** A finite number of distinct, clearly defined states, percepts and actions. Applies also to time.
- **Continuous:** when it is not discrete.

Single or multi-agent task environment

- **Single agent:** an agent operating by itself in an environment.
- **Multi-agent:** it can be competitive or cooperative.

Known or unknown task environment

- **Known:** depends on the knowledge of the agent or the designer of the agent of the rules governing the environment. For each action there is an outcome (if deterministic) or a probability distribution over the possible outcomes (if stochastic).

An environment can also be known and partially observable (for example, a card game) unknown and fully observable (a new videogame).

- **Unknown:** when it is not known.

1.3 Agent functions and programs

An agent is completely specified by the agent function mapping percept sequences to actions. The aim of AI is to design an agent program that implement the agent function concisely.

Definition 7 (Agent program). *They take the current percept as input from the sensors and return an action to the actuator.*

	Chess w clock	Chess no clock	Taxi driving
Fully/partially observable	Fully observable	Fully observable	Partially observable
Deterministic/Stochastic	Deterministic	Deterministic	Stochastic
Episodic/Sequential	Sequential	Sequential	Sequential
Static/Semi/Dynamic	Semi	Static	Dynamic
Discrete/Continuous	Discrete	Discrete	Continuous
Single/Multi-Agent	Multi-agent	Multi-agent	Multi-agent

- The **environment type** largely determines the **agent design**
- The **real world** is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent

Figure 1.4: Task environment example.

Note that the agent program is different from the agent function. The agent program takes the current percept as input, instead the agent function takes the entire percept history.

1.4 Agent types

Types of agent:

- **Simple reflex agents:** select the action on the basis of only the current percept.
- **Model-based reflex agents:** maintain internal state to track aspects of the world that are not evident in the current percept.
- **Goal-based agents:** act to achieve their goals.
- **Utility-based agents:** try to maximize their own expected "happiness".

1.5 How the components of agent programs work

An agent program consists of various components. The components can represent the environment in three ways.

1. Atomic
2. Factored
3. Structured

Atomic representation Each state of the world is indivisible - it has no internal structure.

Factored representation Each state contains a fixed set of variables (or attributes); each variable can a value. Also two different factored states can share some variables.

Structured representation Each state contains objects (with variables with values) and relations with other objects.

Chapter 2

Stable matching problem

2.1 Matching under preferences

The common thread is that: individuals have preferences over the possible outcomes, the task is to find a matching of the participants that is in some sense optimal with respect to these preferences.

- **Two-sided matchings** involving **agents on both sides** (e.g., college admissions, medical resident allocation, job markets, and school choice)
- **Two-sided matchings** involving **agents and objects** (e.g., house allocation, course allocation, project allocation, assigning papers to reviewers, and school choice)
- **One-sided matchings** (e.g., roommate problems, coalition formation games, and kidney exchange)
- **Other recent applications** (e.g., refugee resettlement, food banks, social housing, and daycare)

Figure 2.1: List of topics related to the stable matching problem.

2.2 Stable matching problems

Formulation of the problem: we have two sets of agents, each of them express preferences over agents of the other set.

Goal: choose a matching among the agents of two sets based on their preferences.

Definition 8 (Matching). *A matching is a set of pairs (A_1, A_2) where A_1 comes from the first set, A_2 from the second set.*

2.2.1 Stable Marriage (ideal) formulation

The original formulation of the Stable Matching problem is the Stable Marriage problem. In this scenario we have two sets: one of men and one of women. We will consider the *idealized model* in which we make two assumption that simplify our study:

1. We have the same number of men and women.
2. All men totally order all women, and vice versa.

The list of preferences must be strict (ties are not allowed in this ideal formulation). We will relax those assumptions later.

Definition 9 (Marriage). *A marriage is a one-to-one correspondence between men and women.*

Definition 10 (Blocking pair). *A pair (m, w) where m is a man and w is a woman is a blocking pair if:*

1. *The pairs (m, w') and (m', w) is in the marriage.*
2. *But m prefers w to w' ,*
3. *and w prefers m to m' .*

In other words: m and w are married with other partners but they would prefer to be together.

Definition 11 (Stable marriage). *A stable marriage is a marriage with no blocking pairs.*

- **Given preferences of n men**
 - Greg: Amy>Bertha>Clare
 - Harry: Bertha>Amy>Clare
 - Ian: Amy>Bertha>Clare
- **Given preferences of n women**
 - Amy: Harry>Greg>Ian
 - Bertha: Greg>Harry>Ian
 - Clare: Greg>Harry>Ian
- **Find a stable marriage**

Figure 2.2: Example of preferences lists.

In the figure below, we have an example of unstable marriage. We can see that the pair $(Greg, Bertha)$ is not in the solution set but they would prefer to be together rather

than the partner they are currently with. Also note that in the stable matching solution example, the solution privileges men rather than women: two out of three men are married with their top choice.

$M = \{ (Greg, Amy), (Harry, Bertha), (Ian, Clare) \}$

- Greg: **Amy** > Bertha > Clare
- Harry: **Bertha** > Amy > Clare
- Ian: Amy > Bertha > **Clare**
- Amy: Harry > **Greg** > Ian
- Bertha: Greg > **Harry** > Ian
- Clare: Greg > Harry > **Ian**

Men do ok, women less well

Figure 2.3: Example of stable marriage.

$M = \{ (Greg, Clare), (Harry, Bertha), (Ian, Amy) \}$

- Greg: Amy > Bertha > **Clare**
- Harry: **Bertha** > Amy > Clare
- Ian: **Amy** > Bertha > Clare

Blocking pair:
makes the marriage not stable

- Amy: Harry > Greg > **Ian**
- **Bertha**: Greg > **Harry** > Ian
- Clare: **Greg** > Harry > Ian

Bertha & Greg would prefer to be together

Figure 2.4: Example of unstable marriage.

In general, given our assumptions, any stable marriage problem has at least one stable marriage. Also in general there exist many stable marriages, especially in large domains.

2.3 Gale Shapley algorithm

To use it, the two assumptions made in the idealized formulation of the stable marriage formulation must hold true.

Algorithm 1 Gale-Shapley Algorithm

```

1: Initialize every person to be free
2: while there exists a free man do
3:   Find the best woman he has not proposed to yet
4:   if this woman is free then
5:     Declare them engaged
6:   else
7:     if this woman prefers this proposal to her current partner then
8:       Declare them engaged (and free her current partner)      ▷ Switch
9:     else
10:      This woman prefers her current partner and she rejects the proposal  ▷
11:      Reject
12:    end if
13:  end if
14: end while

```

2.3.1 Features of the algorithm

Properties of the algorithm:

1. Terminates with everyone married.
2. Terminates with a stable marriage.

Proof. We want to prove that the algorithm terminates with a stable marriage. We will prove it by absurd.

Suppose that there is a blocking pair (m, w) not married. The marriage contains instead (m, w') and (m', w) , where m prefers w to w' , and w prefers m to m' .

[Case 1]: m never proposed to w .

This is not possible since, in the algorithm, men move down with the proposals and w' is less preferred than w .

[Case 2]: m had proposed to w .

By hypothesis (m, w) are not married, and since w is preferred to w' , it follows that w rejected m somehow. But, since women only ever trade up, then implies that w prefers m' to m . Therefore the current pairing is stable. Contradiction. So this case can never happen. $\square$

Other properties:

- GS algorithm is $O(n^2)$.
- The definition provided finds **men optimal solution** and **women pessimal solution**.
- If we invert the role of men and women, then the algorithm finds **women optimal solution** and **men pessimal solution**.

In general, a set of agents is favored over the other one with GS algorithm. However there exist algorithms that find more balanced/fairer marriages.

2.4 Formulation extensions

2.4.1 Ties in preferences

We consider the case in which we allow ties in preferences. This is a relaxation of the ideal formulation (in which they are not allowed). We need to introduce two more definition

of stability.

Definition 12 (Weakly stable marriage). *A marriage is weakly stable if there is not an un-matched couple such that each one strictly prefers the other to the current partner (i.e. if there is not a blocking pair).*

Definition 13 (Strongly stable marriage). *A marriage is strongly stable if there is not an un-matched couple such that one strictly prefers ($>$) the other, and the other likes it as much or more ($\geq$) as the current partner.*



In the definition of strongly stable marriage, the definition of blocking pair is enlarged, so it is easy to find blocking pairs. For this reason a strongly stable marriage is a subset of a weakly stable marriage. The definition of weakly stable marriage is equivalent to the one seen for the ideal case.

In general **it doesn't always exist a strongly stable marriage**. Instead **it always exist a weakly stable marriage**. This is easy to prove, we just need to break ties arbitrarily (we need to *linearize*), then we run the GS algorithm which returns a weakly stable marriage (it always finds a solution).

2.4.2 Incomplete preferences

In this subsection we consider another relaxation of the ideal formulation: incomplete lists of preferences are allowed. With this relaxation we introduce some pairs that are unacceptable. We need to extend the definition of blocking pairs.

Note that in this subsection we consider only incomplete preferences lists, instead ties are not allowed.

Definition 14. *A pair (m, w) is a blocking pair if:*

- *m and w are unmatched (i.e.: they aren't married) and do not find each other unacceptable or*

- m, w both prefers each other to the current partners, or
- one of the two is matched (i.e. is married to someone else) but acceptable to the other and prefers the other who is unmatched.

The first case implies that: m and w are not married (with no one) but they find each other acceptable so they should marry each other. The third case implies that: m is married with w' (say) but m is acceptable to w ; m prefers w to w' and also w is unmatched.

The GS algorithm can be easily extended to work with this formulation (some elements can be unmatched).

Theorem 1. *The set of unmatched elements is the same in every stable marriage.*

2.4.3 Ties and incomplete preferences

If we only have ties in preferences, then we have seen that it always exist a weakly stable marriage. Also (from the properties of GS algorithm), the weakly stable marriage is *complete*: all men and women are matched. It follows that in this case the cardinality of the weakly stable marriages is fixed (is maximal).

If we only have incomplete preferences, then weakly stable marriages have a fixed cardinality. This is a consequence of the fact that the unmatched elements are the same in every stable marriage.

If we have both ties and incomplete preferences, then weakly stable marriages may have different sizes.

Theorem 2. *Finding weakly stable marriage of maximal cardinality in the case of ties and incomplete preferences, is a NP-hard problem.*

2.5 Manipulation and strategies

If we analyze the GS algorithm (with male optimality version), we can say that it is **strategy proof for man** since man cannot **manipulate** their preferences in order to get a better solution.

Instead, if we run the male optimality version, women can profit from lying: they can obtain a better partner by expressing different preferences from the true ones (**manipulation**).

Greg: Amy > Bertha > Clare Harry: Bertha > Amy > Clare Ian: Amy > Bertha > Clare Amy: Harry > Greg > Ian Bertha: Greg > Harry > Ian Clare: Greg > Harry > Ian Result of running GS on <u>true prefs</u>	□ Greg: Amy > Bertha > Clare □ Harry: Bertha > Amy > Clare □ Ian: Amy > Bertha > Clare Amy lies □ Amy: Harry > Ian > Greg □ Bertha: Greg > Harry > Ian □ Clare: Greg > Harry > Ian Result of running GS on <u>manipulated prefs</u>
--	---

Figure 2.5: Amy get a better solution if she lies about her preferences.

In general it is not true to say that the manipulations always result on a better solution for someone or for everyone.

Observation 3. *GS algorithm can be manipulated.*

Theorem 4. *Every stable marriage procedure (that is, every procedure that returns a stable marriage) can be manipulated if preference lists can be incomplete.*

Chapter 3

Solving problem by searching

3.1 Problem solving agent

A problem solving agent is a particular type of goal-based agent. This type of agent uses the atomic representation, so each states has no internal structure visible to the problem-solving algorithm. The solution provided by the agent is a **fixed sequence of actions**.

If the environment is observable, known and deterministic, then a solution to the problem is a fixed sequence of actions. The **search** is a process that aims to find such a sequence.

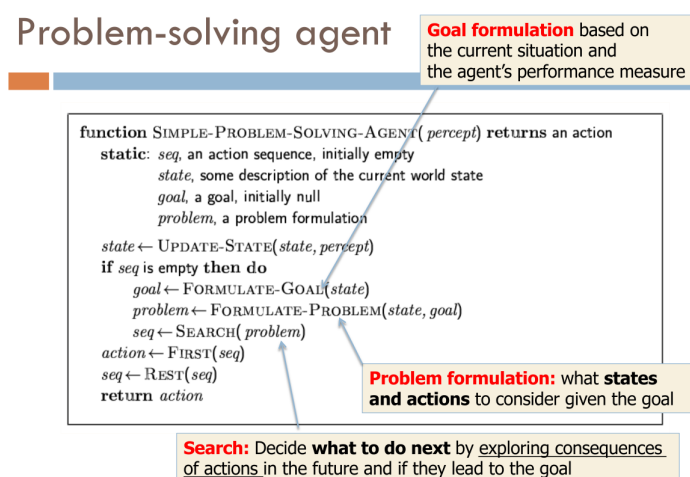


Figure 3.1: Problem-solving agent scheme.

3.2 Problem formulation

A problem can be defined formally by:

- **Initial state** that the agent starts in.
- **Actions** available to the agent. Given a state s , $ACTION(s)$ returns the set of actions that can be executed in s . Each action returned is said to be *applicable* in s .
- **Transition model**: $RESULT(s, a)$ returns the state obtained from doing action a in state s . We call *successor* the state reachable from a given state by a single action.
- **Goal test**: allows to check if state is a goal.
- **Path cost**: numeric value associated to each path reflecting the desired performance measure. We assume that the path costs is additive: sum of step costs.
- **Solution**: a sequence of actions (path) leading from the initial state to a goal state.
- **Optimal solution**: a solution with minimal path-cost.

We define the **state space** as the set of all states reachable from the initial state by any sequence of actions. The state space can be depicted as a directed graph, where:

- nodes represent the states
- edges represent the actions
- a path represent a sequence of states connected by a sequence of actions.

3.3 The Search Tree

The solution of a problem solving agent is a sequence of actions leading from the initial state to the goal state. Search algorithms consider different possible sequence of actions that can be represented with a **search tree**. In a search tree we identify:

- **Root**: the initial state.
- **Nodes**: the states.
- **Branches**: the actions.

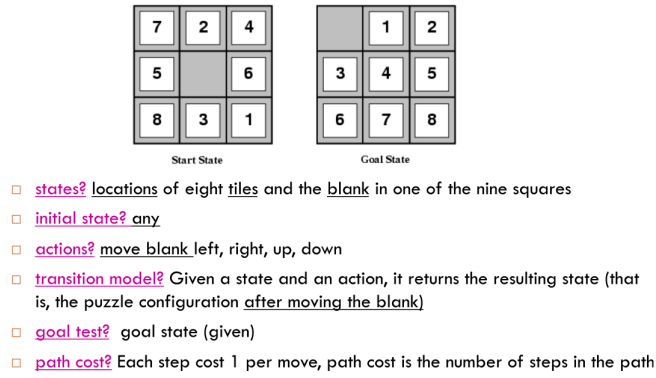


Figure 3.2: Problem formulation result example.

All the outgoing edges from a node represent all the possible available actions starting from that specific state. Note that each state can appear more than once in the search tree.

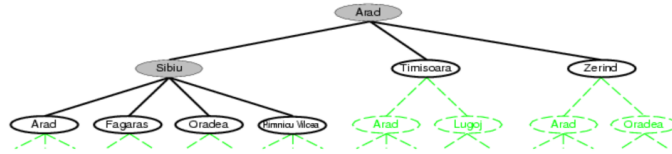


Figure 3.3: Tree search example. The white nodes are leaf node of the tree search.

Definition 15 (Frontier). *The set of all leaf nodes available for expansion at any given point of the algorithm is called frontier.*

Definition 16 (Redundant paths). *Two or more paths are said to be redundant if they both start on state A and end on state B.*

Definition 17 (Loopy path). *A loopy path is a special redundant path that starts and ends on the same state.*

In order to avoid exploring redundant paths, tree search algorithm uses an additional set (**explored set**) that contains all the previously expanded nodes.

3.3.1 Search Tree algorithm infrastructure

Each node is a data structure used to represent the search tree. For each node n we have:

- $n.STATE$: state of the state space to which the node corresponds.

```

function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty

  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
      only if not in the frontier or explored set

```

Figure 3.4: Tree Search Algorithm pseudocode.

- $n.PARENT$: node in the search tree that generated this node.
- $n.ACTION$: the action that was applied to the parent to generate this node.
- $n.PATH-COST$: the cost (denoted with $g(n)$) of the path from the initial state to the node n .

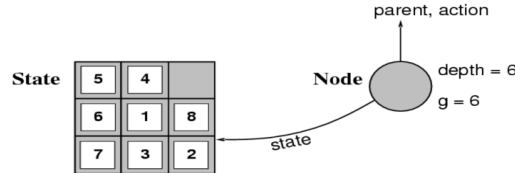


Figure 3.5: Node data structure. The state corresponds to a configuration of the world (it is not a node).

The frontier is implemented with a queue. The type of queue that we use (LIFO, FIFO, priority queue) characterize the **search strategy** of the search tree algorithm.

3.4 Search strategies

Each strategies is defined by picking the order of node expansion. We evaluate each strategy considering:

1. **Completeness**: if it always find a solution.
2. **Time complexity**: number of nodes generated.

3. **Space complexity**: maximum number of nodes in memory.
4. **Optimality**: if it always find a least-cost solution.

Time and space complexity are measured in terms of the branching factor b of the search tree, the depth d of the least-cost solution and the maximum length m of any path in the state space.

3.4.1 Uninformed strategies

Uninformed strategies use only the information available in the problem definition, they don't have any additional information. Some algorithms in this category: *breadth-first search*; *uniform-cost search*; *depth-first search*; *depth-limited search*; *iterative deepening search*.

3.4.2 Informed strategies

Informed strategies use problem-specific knowledge to speed up the search process. The idea is to use an **evaluation function** $f(n)$ which returns an estimate of the *desiderability* of reaching node n . It can be interpreted as a cost estimate function.

The informed search algorithms that we study are all based on the **best-first search algorithm**. This algorithm expands the node in the frontier set that has the lowest $f(n)$. Algorithms that implement best-first search differ in the definition of the function $f(n)$.

Most algorithms use an **heuristic function** $h(n)$ as a component of $f(n)$. This function returns the estimated cost of the cheapest path from node n to a goal state. Assumptions:

1. $h(n)$ is a non negative function.
2. $h(n) = 0$ at every goal state.

Example: $h_{SLD}(n) :=$ the straight line distance from n to Bucharest (goal).

Greedy best-first search The evaluation function is heuristic:

$$f(n) = h(n). \quad (3.1)$$

So the greedy best-first search expands the node that appears to be closest to a goal.

Properties:

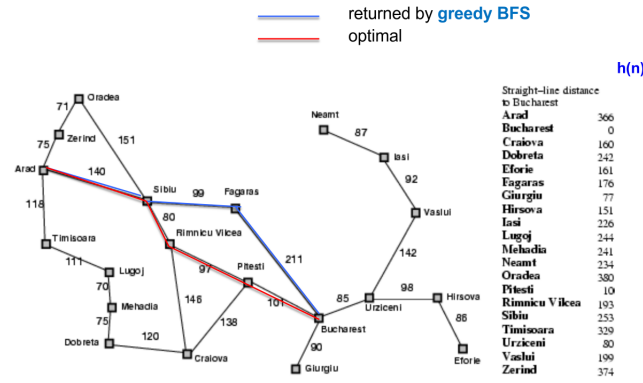


Figure 3.6: Greedy BFS example.

- Complete? No because it can lead to dead end (e.g. ping pong between two states).
If we implement a repeated-state checking then it becomes complete.
- Time: $O(b^m)$
- Space: $O(b^m)$
- Optimal? No.

A-star search The evaluation function is defined as:

$$f(n) = g(n) + h(n) \quad (3.2)$$

where:

- $h(n)$ returns the estimated cost of the cheapest path from n to a goal.
- $g(n)$ returns the path cost from the start node to node n .

So $f(n)$ returns an estimated cost of the cheapest solution passing through node n .

Properties:

- Complete? Yes.
- Time? Exp.
- Space? Keeps all nodes in memory.
- Optimal? Yes if $h(n)$ is an admissible heuristic (see next subsection).

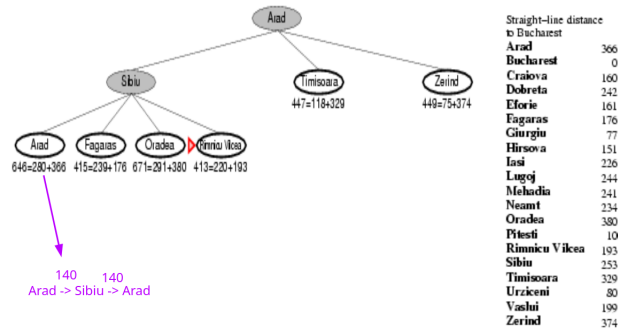


Figure 3.7: A-star example.

3.4.3 Heuristics functions

Definition 18 (Admissible heuristic). *An heuristic function $h(n)$ is said to be admissible if $h(n) \leq h^*(n)$ for every node n , where $h^*(n)$ is the true cost from n to the goal. In other words, an admissible heuristics never overestimates the cost to reach the goal.*

In the previous section, we have used the function $h_{SLD}(n)$. This function was admissible because the straight line distance between two cities is an underestimate of the true distance.

Theorem 5. *If h is admissible, A^* using TREE-SEARCH is optimal.*

Definition 19 (Consistent heuristic). *An heuristic function $h(n)$ is said to be consistent if, for every node n and for every successor n' of n generated by an action a , it holds:*

$$h(n) \leq c(n, a, n') + h(n')$$

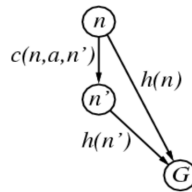


Figure 3.8: Triangular inequality.

One can prove that, if $h(n)$ is consistent $\Rightarrow h(n)$ is admissible.

Definition 20 (Dominance). *Let h_1, h_2 be two admissible heuristic functions. If $h_2(n) \geq h_1(n)$ for all node n , then h_2 dominates h_1 .*

If h_2 dominates h_1 then h_2 is a better choice for a search algorithm because h_1 will expand all the nodes expanded by h_2 and possibly more.

Generating admissible heuristics from relaxed problems A relaxed problem is obtained from the original one by removing some restrictions. In general, in our case, the relaxed problem has a superset of actions available from each state.

Property 6. *Any optimal solution of the original problem is also a solution of the relaxed problem.*

Property 7. *The cost of an optimal solution of the relaxed problem is lower than or equal to an optimal solution of the original problem.*

Observation 8. *The cost of an optimal solution to a relaxed problem is an **admissible heuristic** for the original problem.*

- If the **rules** of the **8-puzzle** are **relaxed** so that
a tile **can move anywhere** (instead of just to the adjacent **empty square**)
then $h_1(n)$ gives the **exact** number of steps of the optimal solution
- If the **rules** of the **8-puzzle** are **relaxed** so that
a tile **can move one square in any direction**, even onto an occupied square,
then $h_2(n)$ gives the **exact** number of steps of the optimal solution

Figure 3.9: Example for the 8-puzzle game.

3.5 Local search algorithms

In many optimization problems we are not interested in the path from the start node to the goal, we only care about the goal state itself which represents the actual **solution**. The **state space** in such cases is defined as a set of "complete configurations", that are all the configurations that do not violate the constraints of the problem.

For such optimization problems, we can use a local search algorithm that basically keep a single "current" state and tries to improve it iteratively. The rule used to update the current state depends on the algorithm used.

A local search algorithm basically explores the **state-space landscape**. A landscape has:

- A location: defined by the state.
- An elevation: defined by the value of the heuristic cost function or objective function.

- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal
 - Each state has n queens on the board, one per column
 - Successors of a state: all possible states generated by moving a single queen to another square in the same column



Figure 3.10: Example: n-queens problem.

The aim is to find:

- A global minimum if elevation corresponds to cost.
- A global maximum if elevation corresponds to objective function.

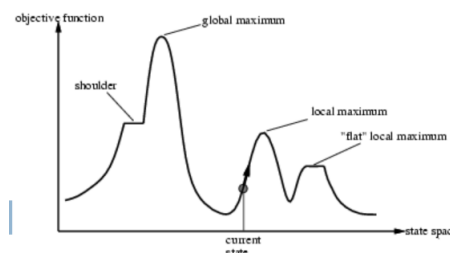


Figure 3.11: State-space landscape.

3.5.1 Hill-climbing search

Assume the elevation corresponds to the objective function.

```

function HILL-CLIMBING(problem) returns a state that is a local maximum
  current ← MAKE-NODE(problem.INITIAL-STATE)
  loop do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE ≤ current.VALUE then return current.STATE
    current ← neighbor

```

Figure 3.12: Hill-climbing search pseudocode.

Rule used: pick a neighbor with the highest value, if there exist many with the same maximum value, then choose at random.

Stop rule: stop when no neighbor has an higher value (local optimal solution).

Problems: gets stuck in local maxima and also on plateaux (flat local maximum or shoulder).

We can consider some variants of the algorithm such as:

- **Stochastic** hill-climbing: chooses at random from the set of all improving neighbors.
- **First-choice** hill-climbing: jumps to the first improving neighbor found.
- **Random-restart** hill-climbing: series of hill climbing runs until a goal is found.

Hill hill-climbing algorithm never makes "downhill" moves toward states with lower value and therefore it get stuck on local maximum. For this reason is said to be **incomplete**. Consider a **purely random walk**: is a **complete** but inefficient algorithm that moves to successor chosen uniformly at random from the st of successor. Combining the two leads to an algorithm that is both efficient and complete.

3.5.2 Simulated annealing search

Simulated annealing is a version of stochastic hill-climbing where some downhill moves are allowed. The idea: allow some "bad" moves to (useful to escape local minima) but gradually decrease their frequency iteration by iteration.

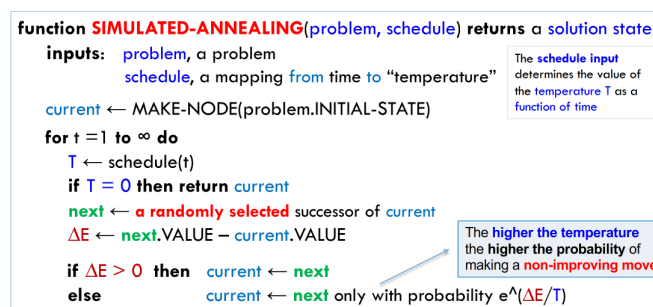


Figure 3.13: Simulated annealing search.

Rule used:

- If the move improves the current state, then go for it.
- Otherwise move with some probability that decreases iteration by iteration.

Property 9. One can prove that: if T decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1.

3.5.3 Local beam search

Keep track of k states rather than just one. Start with k randomly generated states and at each iteration, all the successor of k states are generated. **Rule used:**

- If any of the successor is a goal state, then the algorithm stops.
- Otherwise select the k best successor from the complete list and repeat.

3.5.4 Genetic algorithms

Main features:

- **Population:** start with k randomly generated states called population. Each state is represented as a string over a finite alphabet (often binary strings).
- **Fitness function:** each state is associated to a value via an evaluation function which returns higher values for better states.
- **Update rule:** the next generation of states is produced by selection, crossover and mutation.

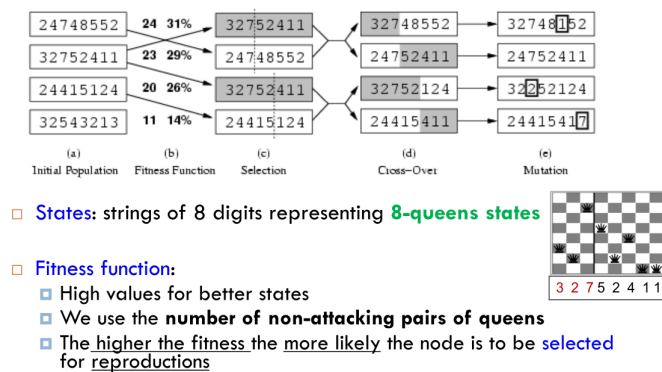


Figure 3.14: Genetic algorithm example for the 8-queens problem. Note that an individual is not selected, and another one is selected twice. The percentage are the probability of being chosen the other value is the one returned by the fitness function.

Chapter 4

Constraint Satisfaction Problems

4.1 Basic definitions

In standard search problems the states were atomic. In CSPs we consider a factored representation for each state. We define:

- **State** is defined by variables X_i with values domain D_i .
- **Goal test** is a set of constraints specifying allowable combinations of values for subsets of variables.
- **Solution**: one value of each variable that satisfies all the constraints.

Algorithms that solves CSPs are more powerful than standard search problems, also they use general-purpose heuristics rather than specific ones.

CSPs solvers are much efficient that search strategies because they eliminate large portions of search space all at once by exploiting the constraints imposed on the variables (**constraints propagation** allows pruning of the search tree).

Three main components of a CSP are:

1. Set of variables $X = \{X_1, \dots, X_n\}$
2. Set of domains $D = \{D_1, \dots, D_n\}$
3. Set of constraints $C = \{c_i = (\text{scope}_i, \text{rel}_i) \forall i = 1, \dots, h\}$ where the scope is a subset of X (variables constrained by c_i), instead the relation tells us which simultaneous assignments of values to variables in the scope are allowed.

Definition 21 (State). A state is defined by an assignment of values to some or all the variables.

Definition 22 (Type of assignment). An assignment can be

- Consistent if it does not violate any constraints
- Complete if every variable is assigned.
- Partial if only some of the variables are assigned.

Definition 23 (Solution). A solution is a consistent and complete assignment.



Figure 4.2: Map-coloring solution.

Figure 4.1: Map-coloring problem.

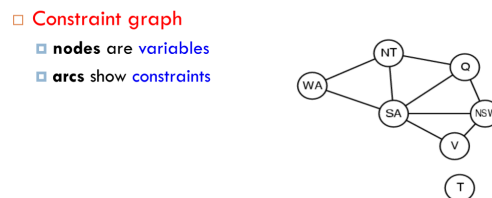


Figure 4.3: Constraint graph of the map-color problem.

4.1.1 Varieties of CSPs

Variables: the simplest kind of CSP involves variables with discrete domains that can be either finite domains or infinite domains. In many real world problems variables are continuous.

Constraints: Constraints in a CSP can be either **unary** ($SA \neq \text{green}$), **binary** ($SA \neq WA$), **higher-order** (more than three variables involved) or **global** (e.g. $Alldiff \rightarrow$ all involved variables must have different values).

Observation 10. Any CSP can be converted into a CSP with only binary constraints (binary CSP).

- Place **one queen** in each column such that **they do not attack** each other
- **Variables:** Q_1, Q_2, Q_3, Q_4 (one per column)
- **Domains:** $D_i = [1, 2, 3, 4]$ (row position of a queen)
- **Constraints:**
 - $Q_i \neq Q_j$ for all i, j (cannot be in the same row)
 - $|Q_i - Q_j| \neq |i - j|$ (cannot be in the same diagonal)

Translate each **constraint** into a set of **allowable values** for its variables

- E.g., values for (Q_1, Q_2) are
(1,3) (1,4) (2,4) (3,1) (4,1) (4,2)

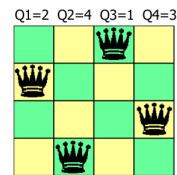


Figure 4.4: CSP 4-queens problem.

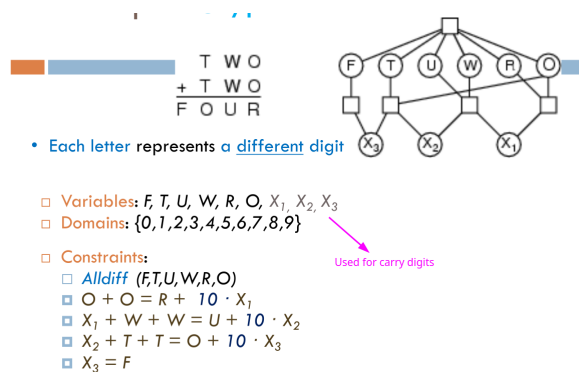


Figure 4.5: Cryptarithmic example.

4.2 Search formulations

4.2.1 Standard search formulation

- **Initial state:** the empty assignment $\{ \}$
- **Successor function:** assign a value to an unassigned variable that does not conflict with current assignment
→ fail if no legal assignments
- **State:** partial assignment
- **Goal test:** the current assignment is complete

Figure 4.6: Standard search formulation.

If we have n variables, then every solution appears at depth n . The algorithm is a DFS.

Problem: consider that we have n variables and each associated domain has size d . Then the branching factor is:

- Level 1: $n \times d$ because we can choose among n variables and each of them has d possible values.
- Level 2: $(n - 1) \times d$
- Level i : $(n - i + 1) \times d$
- Level n : d (choose the value for the last non assigned variable).

So the problem is that the tree has $n! \times d^n$ leaves but the total possible assignments are d^n . This is due to the fact that some assignment combinations are repeated in the search tree (we test multiple times the same solution).

4.2.2 Backtracking search

Backtracking use an (intuitive) trick that avoids to test multiple times the same assignment combination. The **variable assignments are commutative**, therefore we only have to consider a single variable at each level in the search tree. Since at each level the branching factor is d , at the end the tree has d^n leaves (all possible assignments are tested).

```

function BACKTRACKING-SEARCH(csp) returns a solution, or failure
    return BACKTRACK( { }, csp )

function BACKTRACK(assignment, csp) returns a solution, or failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment then
            add {var = value} to assignment
            result ← BACKTRACK(assignment, csp)
            if result ≠ failure then return result
            remove {var = value} from assignment
    return failure

```

Figure 4.7: Backtracking algorithm (DFS).

We call it *backtracking* because when the algorithm finds an invalid assignment, then it restarts from the parent of that node and it considers another value for that variable.

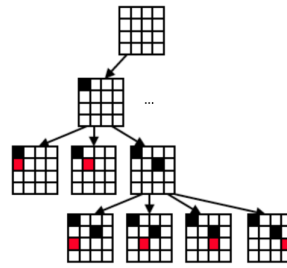


Figure 4.8: Backtracking example (red square = invalid assignment $\Rightarrow$ backtrack).

By using the backtracking algorithm, we have some degree of freedom that we can exploit in order to improve the performances:

1. Choose which variable to assign next.
2. Choose the order of its values.

In the following we consider some general-purpose (heuristics) methods.

Most constrained variable Also called *Fail first heuristic*, chooses the variable with the fewest legal values (also called *minimum remaining values (MRV) heuristic*). The idea behind: we would like to fail fast, going deep in the tree is a waste of time.

Most constraining variable By using the MRV heuristic there might be ties (e.g. at the beginning), we use as a tie-breaker the **most constraining variable** (aka *degree heuristics*) which chooses the variable involved in the most constraints with unassigned variables.



Figure 4.9: At the beginning all the variables have the same amount of remaining values, so we use the degree heuristic to break the tie.

Least constraining value Chooses the variable that rules out the fewest values in the remaining variables (aka *Succeed First*).

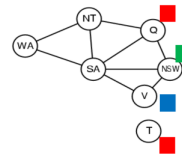


In general, the ordering of the values does not matter if we are interested in finding all the solutions of our problem or if no solution exists. In both cases we need to consider every values.

In the following we consider some improvements to the backtracking search.

4.2.3 Backjumping

As we have seen, the backtracking search backtrack to the previous variable and try another value. This sometimes is a waste of time: if every values of a certain variable leads to an invalid assignment, then backtrack is a useless, it is better to **backjump**.



- **Example:** Assume a **fixed variable ordering** Q, NSW, V, T, SA, WA, NT
 - ▣ Suppose the **partial assignment** $\{Q=\text{red}, NSW=\text{green}, V=\text{blue}, T=\text{red}\}$
 - ▣ The next variable is **SA**, but **every value violates** a constraint
 - ▣ We **back up to T** and try a new color for Tasmania. This is **not useful!**
recoloring Tasmania cannot possibly resolve the problem with **SA**

Figure 4.10: Example in which backtracking is a waste of time (Map-coloring problem). In this case a backjump to variable V is the right choice.

Backjumping jumps to a variable that might fix the problem. The candidate variable is chosen among the ones that belong to a **conflict set** (e.g. in the picture above the conflict set for SA is $\{Q, NSW, V\}$).

4.2.4 No-good

No-good is used to avoid running into the same problem again (vanilla backjumping suffers from this problem). It is a constraint learning technique. **The idea:** finding a minimum set of variables from the conflict set that cause the problem. This set of variables with their corresponding values is called no-good. Then the no-good is added as a new constraint to the CSP (*record* the no-good).

Example

- Consider the state {WA = red, NT = green, Q = blue} (T)
- This state is a **no-good**, because there is no valid assignment to SA
- If the **search tree starts by assigning values for WA, NT, Q** →
recording this no-good would not help
because once we prune this branch from the search tree,
we will **never encounter this combination** again
- If the **search tree starts by assigning values for V, T** →
Useful to record {WA = red, NT = green, Q = blue}
as a **no-good** because we will run into the **same problem**
again **for each possible set of assignments to V and T**

Figure 4.11: No-good example. Sometimes it is useless to record a no-good (no pruning).

From the example we can see that sometimes it is not useful to *record* a no-good. This depends on the variables assignment ordering that we choose.

4.2.5 Forward checking

Backtracking suffers from late detection of failure. To improve this aspect we can use the forward checking (early detection of failure). **Idea:** keep track of remaining legal values for unassigned variables. The search terminates when any variable has no legal values.

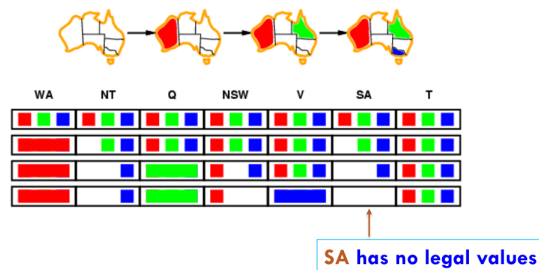


Figure 4.12: Forward checking example. Forward checking as detected that the assignments in last line are inconsistent (WA = red, Q = green, V = blue).

Forward checking propagates information from assigned to unassigned variables. **Cons:** it doesn't provide early detection for **all failures**. For instance, in the previous example, in the third line we can clearly see that NT and SA cannot both be blue.

4.3 Constraint propagation

Constraint propagation is used to reduce the number of legal values for the variables. Is a process that can be used within the search, or in a preprocessing step before search

starts. Sometimes it can solve the whole problem so no search is needed. The idea behind is **local consistency**. By enforcing local consistency (arc consistent, or path consistency, etc) in each part of the constraint graph $\Rightarrow$ inconsistent values are eliminated in graph and therefore we get an easier CSP.

4.3.1 Node consistency

Definition 24 (Node consistency). *A variable (corresponding to a node in the CSP network) is node-consistent if all the values in the variable's domain satisfy the variable's unary constraints.*

Example (a variant of map coloring)

- Assume South Australians **dislike green**
- Variable **SA** starts with domain {red,green,blue}
- We can **make SA node consistent** by **eliminating green**, leaving SA with the reduced domain {red,blue}

Figure 4.13: Node consistency example.

If every variable in the network is node-consistent, then the **CSP network** is said to be **node-consistent**.

Observation 11. *Unary constraints can be eliminated by running node consistency. Also, since all n -ary constraints can be reduced into binary constraints, then we will consider only CSPs with binary constraints.*

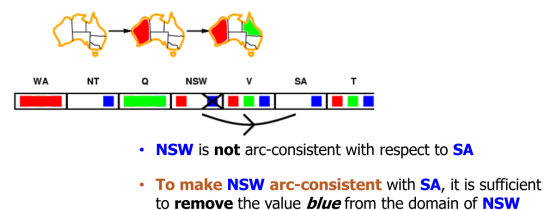
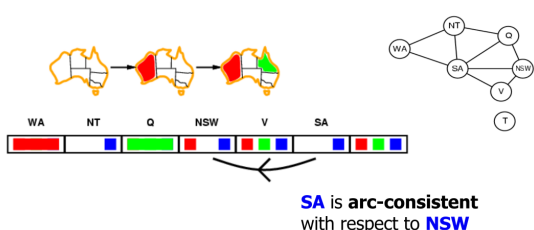
4.3.2 Arc consistency (AC)

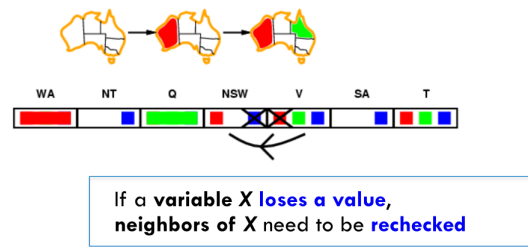
The simplest form of propagation makes each arc consistent.

Definition 25 (Arc consistency). *A variable is arc-consistent if every value of its domain satisfies the variable's binary constraints. Formally: X_i is arc-consistent w.r.t. X_j iff for every value x of X_i , there is some allowed y for X_j .*

A **network** is said to be **arc-consistent** if every variable is arc-consistent with every other variable.

- X_i is **arc consistent** with respect to X_j iff
for every value x for X_i , there is some allowed y for X_j



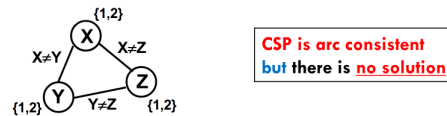


Arc consistency can be run as a preprocessor or after each assignment. In the example above we have used it after an assignment.

Pros: AC detects failure earlier than forward checking. AC is useful to prune the search space. If the all the domain set are singleton, then AC provides a solution.

Cons: arc-consistency removes many incompatible values, however (in general) it doesn't provide a solution and also it doesn't tell us if a solution exists.

Example:



4.3.3 Path consistency (PC)

Requires consistency over more than one constraint.

Definition 26 (Path consistency). A two-variable set $\{X_i, X_j\}$ is path-consistent w.r.t. a third assignment X_m if:

- for every assignment $\{X_i = a, X_j = b\}$ consistent with the constraints on $\{X_i, X_j\}$
- exists at least one assignment to X_m that satisfies
 1. the constraints on $\{X_i, X_m\}$ and
 2. the constraints on $\{X_m, X_j\}$.

Hint: is called path consistency since it is like to consider a path from X_i to X_j with X_m in the middle.

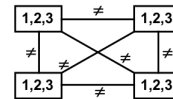
Note that: path consistency does not guarantee that all the constraints among the variables on the path are satisfied; only the constraints between the neighbouring variables must be satisfied.

PC is still **not a complete** technique

$A, B, C, D \in \{1, 2, 3\}$

$A \neq B, A \neq C, A \neq D, B \neq C, B \neq D, C \neq D$

is PC but has not solution



Other stronger consistency notions...

4.4 Local search for CSPs

The point of Local Search is to eliminate violated constraints. Assume an assignment is **inconsistent**. The next one can be constructed in such a way that constraint violation is *smaller*. This assignment should be **better** than the previous one, meaning that more constraints are satisfied. Assignments are not necessarily generated systematically, in this way we lose completeness but we (hopefully) get better efficiency.

Local space terminology:

- **Search space:** set of all complete variable assignments.
- **Set of solutions:** it's a subset of the search space, all complete assignments satisfying all the constraints.
- **Neighborhood relation:** indicating what assignments can be reached by a search step given the current assignment during the search procedure.
- **Evaluation function:** mapping each assignment to a real number representing "how far the assignment is from being a solution".
- **Initialization function:** returns an initial position given a possibility distribution over the assignments.
- **Step function:** Given an assignment, its neighborhood, and the evaluation function, returns the new assignment to be explored by the search.
- **Set of memory states** (optional): holding information about the state of the search mechanism.
- **Termination criterion:** stopping the search when satisfied.

The different algorithms are characterized by the step function and use of memory.

4.4.1 Min-Conflict Local Search

Definition 27 (Conflict set). *The conflict set of a given assignment is defined as the set of variables involved in some constraint that the assignment is violating.*

The Min-Conflict LS procedure:

1. Starts at a **randomly generated assignment**.
2. At each step of the search:
 - (a) **Selects variable** from current conflict set.
 - (b) **Assign a value** that minimizes the number of violated constraints (multiple choices $\Rightarrow$ choose randomly).

4.4.2 Deal with local minima

The evaluation function can have:

- **Local minimum:** a state that is not minimal and there is no state with better evaluation in its neighbourhood¹.
- **Strict local minimum:** a state that is not minimal and there are only states with worse evaluation in its neighbourhood.
- **Global minimum:** the state with the best evaluation.
- **Plateau:** a set of neighbouring states with the same evaluation.

In the following we consider three techniques for preventing stagnation on local minima.

Restart Re-initialize the search after n (non-strictly improving) steps. The new assignment is chosen randomly. This technique can be combined both with hill-climbing and Min-conflicts. It is effective if number n is chosen correctly.

Random walk Add some "noise" to the algorithm by selecting a new assignment from the neighborhood randomly. This technique can hardly find a solution so it needs some guidance. It can be combined with the heuristic guiding the search via probability distribution.

- $p :=$ probability of using random walk;
- $(1 - p) :=$ probability of using heuristic guide.

¹Neighborhood = different values for the selected variable.

Tabu search Being trapped in local minimum can be seen as **cycling**. In order to avoid cycles we can remember already visited states and do not visit them again. To limit memory consumption $\rightarrow$ remember just a few last states (prevents *short* cycles). The k last visited states (forbidden states) are stored in a so called **tabu list**.

4.4.3 Constraints weighting

Technique used to improve the search. Can help concentrate the search on important constraints.

Strategy:

1. **Initialize** each weight to 1.
2. At each step of the search:
 - (a) **Choose a variable-value pair** to change with lowest total weight of all violated constraints.
 - (b) **Adjust weights** by incrementing the ones of each constraint violated by the current assignment.

- It can be **used** for **scheduling problems** in **online setting** when the problem changes
- **A week's airline schedule**
 - may involve thousands of **flights**
 - may involve tens of thousands of **personnel assignments**
 - **bad weather** at one airport can make **the schedule infeasible**
- **To repair the schedule** with a **minimum number of changes**
 - a **local search algorithm** starting from the **current schedule**
- A **backtracking search** with the new set of constraints usually requires
 - much **more time**
 - might find a solution with **many changes** from the **current schedule**

Figure 4.14: Local search in real world problems.

4.5 Structure of the problem

Sometimes we can exploit the problem structure to find solutions quickly. **Idea:** decompose the problem into independent sub-problems. **Independence** can be obtained by finding connected components of the *constraint graph*. If assignment S_i is a solution of subproblem CSP_i , then $\cup_i S_i$ is a solution of $CSP = \cup_i CSP_i$.

- Assume **n variables**, each variable has a domain with cardinality **d**
- Assume **c** is a constant, **$c < n$**
- Assume **each CSP _{i}** has **c variables** from the total of **n** $\rightarrow$ **n/c subproblems**
- Each subproblem requires at most **d^c** work to solve it $\rightarrow$
- Total work is **$O(d^c n/c)$** , which is **linear in n**
- **Without the decomposition**
Total work is **$O(d^n)$** , which is **exponential in n**
- **Example:**
 - Dividing a **Boolean CSP** with **80 variables** into **4 subproblems**
 - Worst-case solution time: from **the lifetime of the universe** to **less than 1 second**

Figure 4.15: Why decomposition is important.

4.5.1 Tree-structured CSP

Tree-structured CSP are easy to solve.

Definition 28. A constraint graph is a tree when any two variables are connected by only one path.

Theorem 12. Any Tree-structured CSP can be solved in time linear in the number of variables.

Definition 29 (Directed arc-consistency). A CSP is defined to be directed arc-consistent (DAC) under an ordering of variables $X_1, X_2, \dots, X_n$ iff every X_i is arc-consistent with each X_j for $j > i$.

If the CSP has n variables with d values in each domain, then we can make it DAC in $O(nd^2)$ (linear in n).

Solve a Tree-structured CSP:

1. Pick any variable to be the **root of the tree**.
2. **Apply topological sort:** choose an ordering of the variables such that each one appears after its parent in the tree.
3. **Make the graph directed arc-consistent.**
4. Follow the list of variables (starting from the root) and **choose any remaining variables**.

DAC guarantees that for any value we choose for the parent, there will be a valid value left to choose for the child.

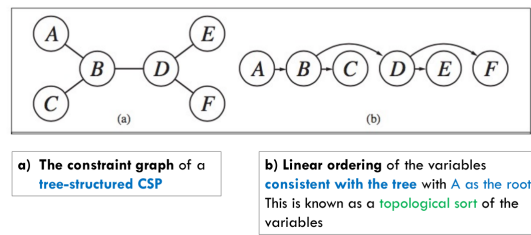


Figure 4.16: Tree-structured CSP example.

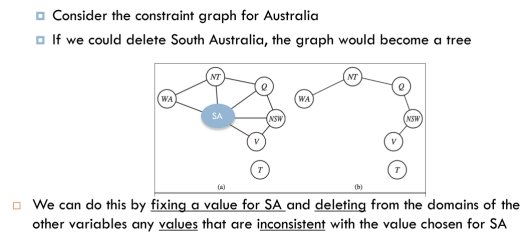


Figure 4.17: Reduction to a tree-structured graph example.

4.5.2 Almost tree-structured

The idea is to reduce the graph structure to a tree, by assigning values to some variables. This technique is called **cutset conditioning**:

1. Choose a subset S (called **cycle cutset**) of the variables such that the **constraint graph becomes a tree** after removal of S .
2. For each possible assignment of variables in S that satisfies all constraints on S :
 - (a) **Remove** from the domains of the remaining variables **any values that are inconsistent** with the assignment for S .
 - (b) If the remaining CSP has a solution, return it together with the assignment for S .

4.5.3 Tree decomposition

Another strategy is the so called tree decomposition. The idea is to **decompose** problem **into a set of connected sub-problems**, where two sub-problems are connected when they share a constraint. Each sub-problem is solved independently and finally the solutions are **combined to get a global solution**.

Property 13 (Tree decomposition). *A tree decomposition must satisfy the following conditions:*

- Every variable of the original CSP appears in at least one sub-problem.
- If two variables are connected by a constraint in the original CSP, then they must appear with their constraint in at least one sub-problem.
- If a variable appears in some sub-problems, then it must have the same value in everyone.

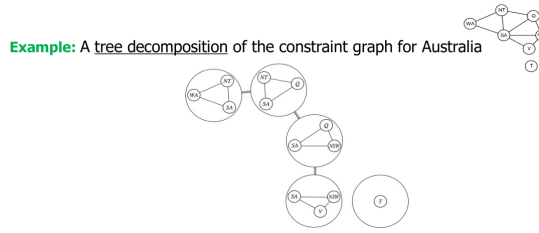


Figure 4.18: Tree decomposition example.

If a sub-problem has no solution, then the entire problem has **no solution**. Instead, if we manage to solve all sub-problems, we can construct a **global solution** as follows:

1. Consider each sub-problem as a new **mega-variable**. The domain is defined as all the solutions of the associated sub-problem.
2. Solve the constraints that connect the sub-problems by using **tree CSP solver**. In this way we construct an overall solution with identical values for the same variable.

In general, a constraint graph allows for **several tree decomposition**. We aim to find the one with the sub-problems as small as possible.

Definition 30. The tree width of a tree decomposition is defined as $s - 1$ where s is the size of the largest sub-problem.

Definition 31. The tree width of a graph is defined as the minimum tree width among all its tree decompositions.

Theorem 14. If a graph has tree-width w and we know the corresponding decomposition, then we can solve the problem in $O(nd^{w+1})$.

Theorem 15. Finding a tree decomposition with minimal tree width is NP-hard.

4.6 Soft CSP

Definition 32 (Soft constraint). A soft constraint is a classical constraint, where each assignment of values to its variables has an associated preference value from a (totally or partially ordered) set. Mathematically is defined as a pair $c = \langle f, \text{con} \rangle$ where:

- con is the **scope**

$$\text{con} = \{X_1^c, \dots, X_k^c\}, \quad \text{con} \subset X$$

- f is the **preference function**

$$f : D(X_1^c) \times \dots \times D(X_k^c) \rightarrow A$$

$$(v_1, \dots, v_k) \rightarrow p$$

where $D(\cdot)$ indicates a domain of a variable, p is a preference value.

Definition 33 (Hard constraint). A hard constraint is a soft constraint where for each tuple $t = (v_1, \dots, v_k)$ we have $f(t) = 1$ (tuple is allowed) or $f(t) = 0$ (forbidden).

This set has two operations ($+$ and $\times$) which makes it similar to a **semiring**. We call it **c-semiring** (where the 'c' stands for *constraint*). The preference value indicates the *desirability* of the associated assignment.

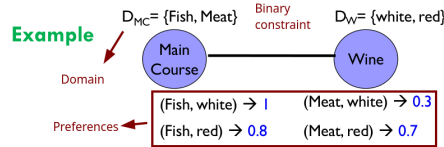


Figure 4.19: Soft CSP example.

Definition 34 (c-semiring). A c-semiring is defined as a tuple $S = \langle A, +, \times, \mathbf{0}, \mathbf{1} \rangle$ where:

- A is a set that specifies the preference values to be associated with each tuple (i.e. with each assignment of values for the variables).
- $+$ is the **additive operator**. It induces the ordering $\leq_S$ defined as

$$a \leq_S b \text{ iff } a + b = b$$

that means "b is better than a if and only if the $+$ operator returns b".

- $\times$ is a multiplicative operator that combines the preferences of different constraints.
- 0 is the worst preference value, $0 \in A$.
- 1 is the best preference value, $1 \in A$.

Definition 35 (Soft CSP). A soft CSP is defined by a set of soft constraints over a set of variables based on a specific c-semiring $\langle A, +, \times, 0, 1 \rangle$. A **solution** is defined by a complete assignment and the **preference value of a solution** is computed by combining (via $\times$) the preference values of the partial assignments.

A soft CSP induces an **ordering over the solutions**, from the ordering $\leq_S$ of the c-semiring. The ordering $\leq_S$ can be a **total order** over solutions or a **partial order**.

In general there exist different possible instances of soft CSPs, each characterized by a c-semiring. In the following we introduce two of them.

4.6.1 Fuzzy CSP

The fuzzy c-semiring is defined as:

$$\langle A = [0, 1], + = \max, \times = \min, 0 = 0, 1 = 1 \rangle.$$

We can see that the preferences are values between 0 and 1; higher values denote better preferences (because of the definition of $+$); 0 is the worst preference, 1 is the best; combination is taking the smallest value.

The optimization criterion used is *maximize the minimum preference* (it is derived from the definition of the two operators $+$, $\times$). This is a pessimistic approach which can be useful in critical applications.

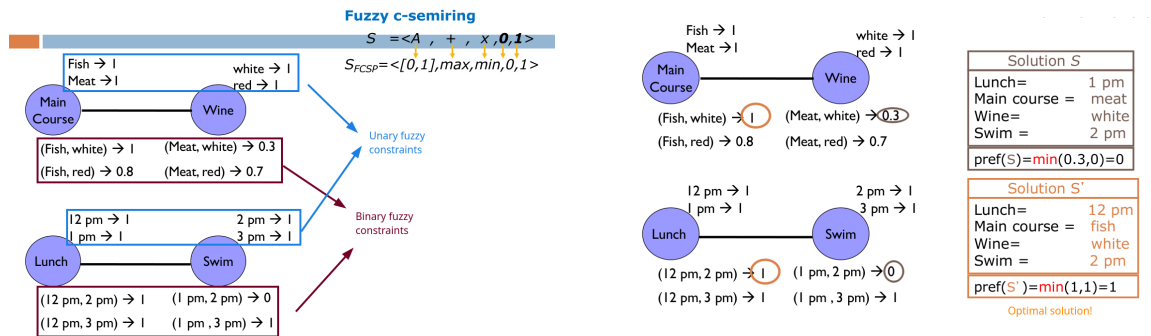


Figure 4.20: Fuzzy CSP example.

4.6.2 Weighted CSP

The weight c-semiring is defined as:

$$\langle A = [0, +\infty], + = \min, \times = +, \mathbf{0} = +\infty, \mathbf{1} = 0 \rangle.$$

In this case, lower costs are better (from definition of $+$ operator; the worst cost is $+\infty$ and the best one is 0. We can see that combination is taking the sum of the costs.

The optimization criterion used is *minimize the sum of costs*. This can be useful in all settings where costs apply.

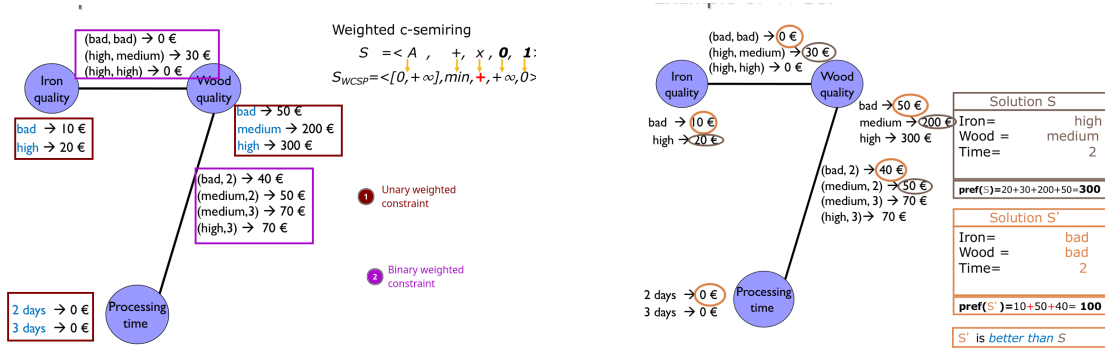


Figure 4.21: Weighted CSP example.

4.6.3 Other soft CSPs

Each instance is characterized by a c-semiring $\langle A, +, \times, \mathbf{0}, \mathbf{1} \rangle$

- **Classical CSPs:** $\langle \{0,1\}, \text{logical or}, \text{logical and}, 0, 1 \rangle$
 - Satisfy all constraints
- **Fuzzy CSPs:** $\langle [0,1], \max, \min, 0, 1 \rangle$
 - Maximize the minimum preference
- **Weighted CSPs:** $\langle [0,+\infty], \min, +, +\infty, 0 \rangle$
 - Minimize the sum of the costs
- **Probabilistic CSPs:** $\langle [0,1], \max, \times, 0, 1 \rangle$
 - Maximize the joint probability
- **Multi-criteria problems:** Cartesian product of c-semirings
- ...

4.6.4 Multi-criteria problems

If we're considering multiple and different criteria, then we need to define one c-semiring for each one.

Given n c-semiring $S_i = \langle A_i, +_i, \times_i, \mathbf{0}_i, \mathbf{1}_i \rangle$ for $i = 1, \dots, n$ we can build the c-semiring S as follows:

$$\langle \langle A_1, \dots, A_n \rangle, +, \times, \langle \mathbf{0}_1, \dots, \mathbf{0}_n \rangle, \langle \mathbf{1}_1, \dots, \mathbf{1}_n \rangle \rangle$$

Each assignment is associated with a **tuple of preference values** $(p_1, \dots, p_n)$. The ordering over the solutions may be a partial order.

Problem: choosing a route between two cities

□ Each piece of highway has

- a **preference** and
- a **cost**

□ **Two goals:** we want

- **minimize the sum of the costs** and
- **maximize the preference**

□ **C-semiring:** by putting together

- one **fuzzy c-semiring** $\langle [0, 1], \max, \min, 0 \rangle$
- one **weighted c-semiring** $\langle [0, +\infty], \min, +, +\infty, 0 \rangle$

□ **Best solutions:** routes such that there is no other route with a better semiring value

□ $\langle 0.8, \$10 \rangle$ is better than $\langle 0.7, \$15 \rangle$

□ **Order over solutions** is **partial**

□ $\langle 0.6, \$10 \rangle$ and $\langle 0.4, \$5 \rangle$ are not comparable

Figure 4.22: Weighted CSP example.

4.6.5 Fundamental operations

Projection The projection operation eliminates one or more variables from a constraint obtaining a new soft constraint preserving the information on the remaining variables.

Formally:

Definition 36 (Projection). *Given a soft constraint $c = \langle f, \text{con} \rangle$, a set I of variables ($I \subset X$), the projection of c over I is the constraint $c|_I = \langle f', \text{con}' \rangle$ where*

- $\text{con}' = I \cap \text{con}$
- $f'(t') = +(f(t))$ over tuples of values t s.t. $t|_{I \cap \text{con}} = t'$.

Combination The combination operation combines two or more soft constraints obtaining a new soft constraint "synthesizing" the information of the original ones.

Formally:

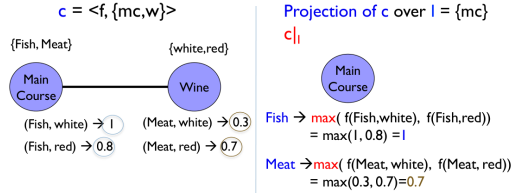
Definition 37 (Combination). *Given two soft constraints $c_1 = \langle f_1, \text{con}_1 \rangle$ and $c_2 = \langle f_2, \text{con}_2 \rangle$, then their combination is the constraint $c_1 \times c_2 = \langle f', \text{con}' \rangle$, where*

- $\text{con}' = \text{con}_1 \cup \text{con}_2$
- $f'(t) = f_1(t|_{\text{con}_1}) \times f_2(t|_{\text{con}_2})$

Projection: fuzzy example

$$S_{FCSP} = \langle [0, 1], \text{max}, \text{min}, 0, 1 \rangle$$

If $c = \langle f, \text{con} \rangle$, then $c|_l = \langle f, l \cap \text{con} \rangle$
 $f(t) = + (f(t))$ over tuples of values t s.t. $t|_{l \cap \text{con}} = t'$



Projection: weighted example

$$S_{WCSP} = \langle [0, +\infty], \text{min}, +, +\infty, 0 \rangle$$

If $c = \langle f, \text{con} \rangle$, then $c|_l = \langle f, l \cap \text{con} \rangle$
 $f(t) = + (f(t))$ over tuples of values t s.t. $t|_{l \cap \text{con}} = t'$

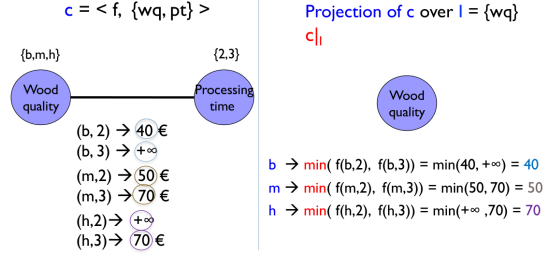
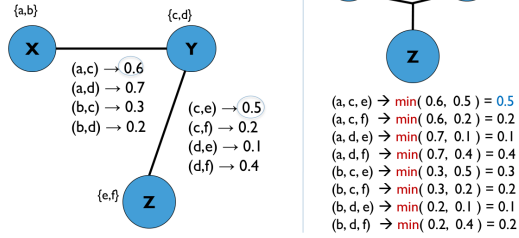


Figure 4.23: Projection examples.

Combination: fuzzy example

$$S_{FCSP} = \langle [0, 1], \text{max}, \text{min}, 0, 1 \rangle$$

If $c_1 = \langle f_1, \text{con}_1 \rangle$ and $c_2 = \langle f_2, \text{con}_2 \rangle$ then: $c_1 \times c_2 = \langle f, \text{con}_1 \cup \text{con}_2 \rangle$
 $f(t) = f_1(t|_{\text{con}_1}) \times f_2(t|_{\text{con}_2})$



Combination: weighted example

$$S_{WCSP} = \langle [0, +\infty], \text{min}, +, +\infty, 0 \rangle$$

If $c_1 = \langle f_1, \text{con}_1 \rangle$ and $c_2 = \langle f_2, \text{con}_2 \rangle$ then: $c_1 \times c_2 = \langle f, \text{con}_1 \cup \text{con}_2 \rangle$
 $f(t) = f_1(t|_{\text{con}_1}) \times f_2(t|_{\text{con}_2})$

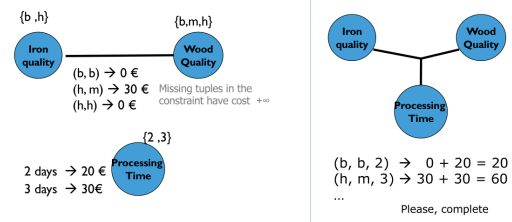


Figure 4.24: Combination examples.

4.6.6 How to find optimal solutions

In general stating if a solution t is better than another solution t' is easy (linear in number of constraints) since we just need to compute two preference levels and compare them. In the other hand, finding an optimal solution is not an easy task.

To find optimal solutions we can use the **Branch and Bound** algorithm which adopts a backtracking strategy (**systematic search**). The idea is to visit each assignment that *may be a solution* and skip only the ones that are shown to be *dominated by others*.

Consider a c-semiring $\langle A, +, \times, \mathbf{0}, \mathbf{1} \rangle$. Then we define:

- **Lower bound** (global): the preference of best solution so far ($\mathbf{0}$) at the beginning.
- **Upper bound** for each node: upper bound to the preference of any assignment in the subtree rooted at the node. Is computed as *combination* of preferences from

constraints on assigned variables.

- If ub is worse than lb the subtree is pruned.

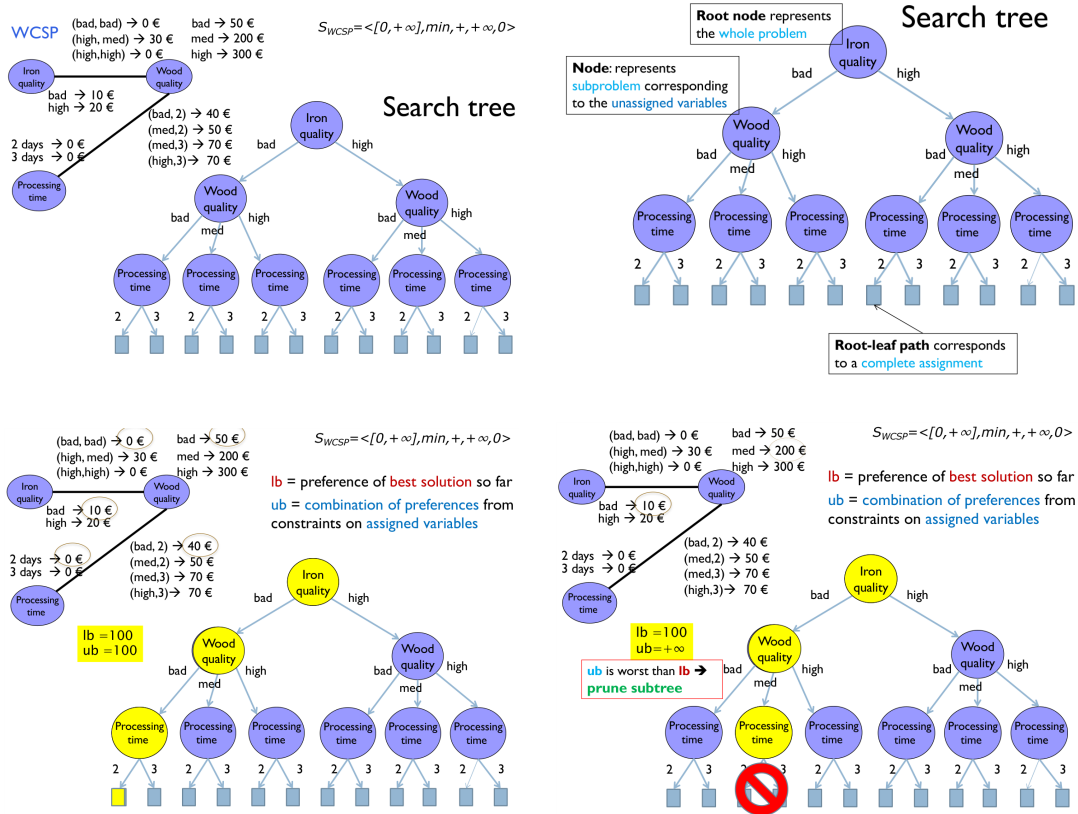


Figure 4.25: Branch and Bound example for a WCSP. See full example in slides.

Chapter 5

Multi-agent decision making

5.1 Defining preferences

Definition 38 (Preferences). *Preferences are orderings over a set possible options and can be used to model levels of acceptance or costs.*

Preferences are NOT constraints. Indeed constraints are strict requirements that must be satisfied, instead preferences are more tolerant (**tolerant constraints**). Tolerant constraints allow for some degree of flexibility and can accommodate a range of acceptable solution. Both preferences and constraints may be present in the same problem (e.g. timetabling).

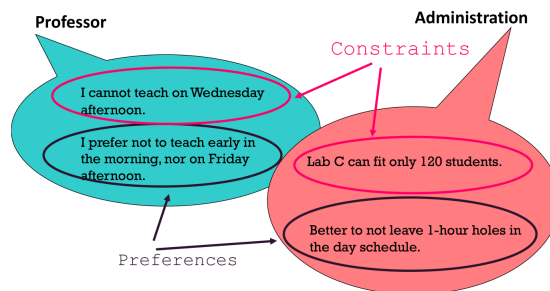


Figure 5.1: University timetable: preferences and constraints.

There exist several kinds of preferences:

- **Unconditional** (e.g. *I prefer taking the bus*).
- **Conditional** (e.g. *I prefer taking the bus if it's raining*).
- **Multi-agent** (e.g. *I like blue, my husband likes green, what color for the new car?*)

- **Quantitative:** Numbers, or ordered set of objects (e.g. *My preference for ice cream is 0.8, and for cake is 0.6*).
- **Qualitative:** Pairwise comparison (e.g. *'Ice cream is better than cake'*).

Two compact way to model preferences are the following:

1. **Soft constraints:** for modeling *quantitative* and *unconditional* preferences.
2. **CP-nets** for modeling *qualitative* and *conditional* preferences.

5.2 Preferences in multi-agent systems

Consider a scenario in which we have several agents, a common set of possible decisions and each agents has its preferences over the possible decisions. **Goal:** to choose one of the decisions based on the preferences of the agents, or a set of decisions, or a ranking over the decisions.

How can we compute a collective decision? Let the agents vote by expressing their preferences over the possible decisions and then **aggregate** the votes **to get a single decision**. In the following section we will introduce the *voting theory* which will give a more detailed answer to this topic.

5.3 Voting theory

In voting theory:

- Agents $\equiv$ **Voters**.
- Decisions $\equiv$ **Candidates**.
- Preferences remains preferences.
- Chosen decision $\equiv$ **Winner**.

As we have already said in the previous section, the goal is to find one candidate (the winner) based on the voters' preferences. To do so we need to define a **voting rule**. There exist many of them, in the following we introduce some of them.

- **Plurality**
 - **Voting:** each voter provides the most preferred decision.

- **Selection:** the decision preferred by the largest number of voters.
- **Majority**
 - **Voting:** similar to plurality voting, each voter selects their most preferred option.
 - **Selection:** the option that receives more than half of the votes is selected. If no option receives more than half of the votes, a runoff or another method is used to determine the winner. This rule is typically applied when there are only two options, making it straightforward to determine the majority preference.
- **Approval** (m options)
 - **Voting:** each voter approves any number of options.
 - **Selection:** the option with the most votes.
- **Borda**
 - **Voting:** each voter provides a ranking over all options.
 - **Score of an option** for a voter: the number of options that it dominates.
 - **Selection:** the option with the greatest sum of scores.

rank	rank	rank	rank	rank	Borda Count	Winner
3	3	3	3	3	9	
2	2	2	2	2	8	
1	1	1	1	1	7	
0	0	0	0	0	6	
Voter 1	Voter 2	Voter 3	Voter 4	Voter 5		

Figure 5.2: Borda example.

Some desirable properties that we aim to achieve:

Property 16 (Unanimity). *If all voters have the same top choice, is selected.*

Property 17 (Non-dictatorship). *There is no dictator (i.e., a voter such that his top choice always wins, regardless of the votes of other voters).*

Property 18 (Non-manipulability). *There is no incentive for agents to misrepresent their preferences.*

Two classical impossibility results:

Theorem 19 (Arrow’s theorem). *In totally ordered preferences, it is impossible to find a voting rule with some desirable properties including unanimity and non-dictatoriality.*

Theorem 20 (Gibbard-Sattherwaite’s theorem). *In totally ordered preferences, it is impossible to have a voting rule that is non-dictatorial and non-manipulable.*

These impossibility results hold also when we allow partially ordered preferences

5.4 Multi-agent AI scenarios

In multi-agent AI scenarios, we usually need to deal with different problems.

Incomparability Preferences do **not always** induce a **total order** over the options and may induce a **partial order** where some options are incomparable. In general some options are naturally incomparable.

Also an agent may have several possibly **conflicting preference criteria** he wants to follow (e.g.: a cheap and slow car is incomparable w.r.t. an expensive and fast car).

Uncertainty, vagueness There would be cases where there are some missing preferences, due to high cost to compute it or due to privacy concerns or imprecise concerns, s.t. preferences coming from sensor data or preferences computed in order to reduce the cost for the exact preference.

The Aim of AI: find compact preference formalisms and solving techniques to model and solve problems with missing or imprecise preferences.

Computational concerns In the context of multi-agent systems, efficient resource management is essential, as certain tasks —such as modeling agents’ preferences, computing the winner, or reasoning over these preferences— can be computationally expensive. While efficiency plays a key role, it is equally important to safeguard the integrity of the decision-making process by ensuring it remains robust against manipulation.

5.5 Formalisms to model compactly preferences

The goal is to find a compact preference formalism to express agents’ preferences.

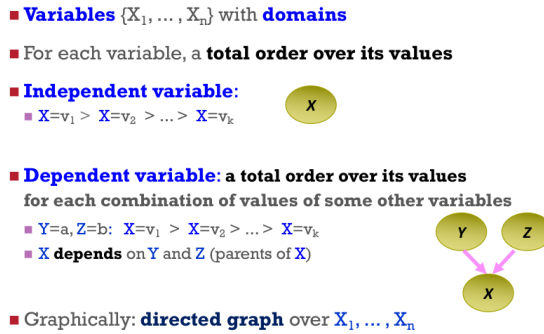
5.5.1 Soft constraints

The first model uses Soft constraint formalism (the c-semiring framework) where the agent expresses his preferences over partial assignments of the decision variables. From these preferences the preference ordering over the solution space is generated.

As mentioned before, soft constraints model *quantitative* and *unconditional preferences*. In general it is difficult to elicitate quantitative preferences from user and also many problems conditional statements. To overcome this limitation we can use the following formalism.

5.5.2 CP-net

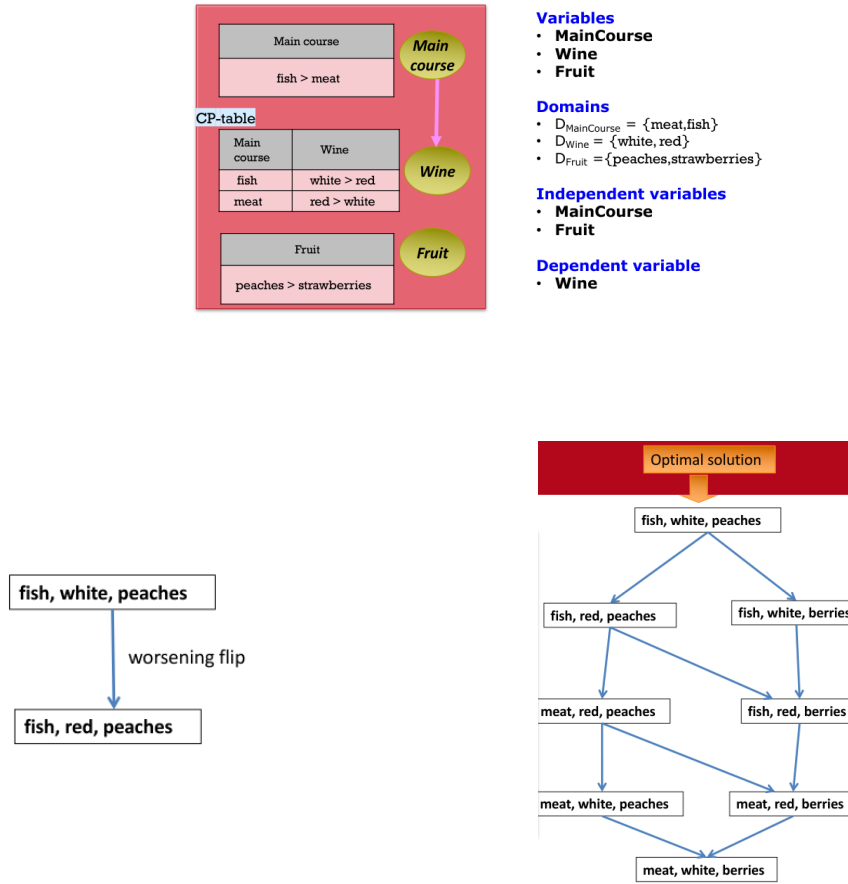
CP-net stands for *Conditional Preference network*. It is a formalism used to compactly represent qualitative and conditional preferences.



Definition 39 (CP-net). A CP-net over variables $\mathbf{V} = \{X_1, \dots, X_n\}$ is

- a directed graph G over $X_1, \dots, X_n$
- whose nodes are annotated with conditional preference tables $\text{CPT}(X_i)$ for each $X_i \in \mathbf{V}$.

Each conditional preference table $\text{CPT}(X_i)$ associates a total order $\succ$ with each instantiation $\mathbf{u}$ of X_i 's parents $\text{Pa}(X_i) = \mathbf{U}$.



Definition 40 (Worsening flip). We call *worsening flip* the change of the value of a variable in a way that is less preferred in some statement. An outcome O_1 is preferred to O_2 if and only if there is a sequence of worsening flip from O_1 to O_2 .

An outcome is **optimal** if no other outcome is preferred. In general, if the CP-net is acyclic, that it is easy to find an optimal solution.

1. First consider independent variables and assign them their most preferred values.
2. Then consider dependent variables, that directly depend on the assigned variables and assign them their most preferred values that are consistent with the values previously assigned to their parents.
3. And so on until we assign a value to all the variables.

5.6 Sequential voting

There are several voters and decisions are made by several issues. **Main idea:** vote separately on each issue, but do so sequentially. This gives voters the opportunity to

	Soft CSPs	Tree-like soft CSPs	CP-nets	Acyclic CP-nets
Preference orderings	all	all	some	some
Find an optimal decision	difficult	easy	difficult	easy
Compare two decisions	easy	easy	difficult	difficult
Check if a decision is optimal	difficult	easy	easy	easy

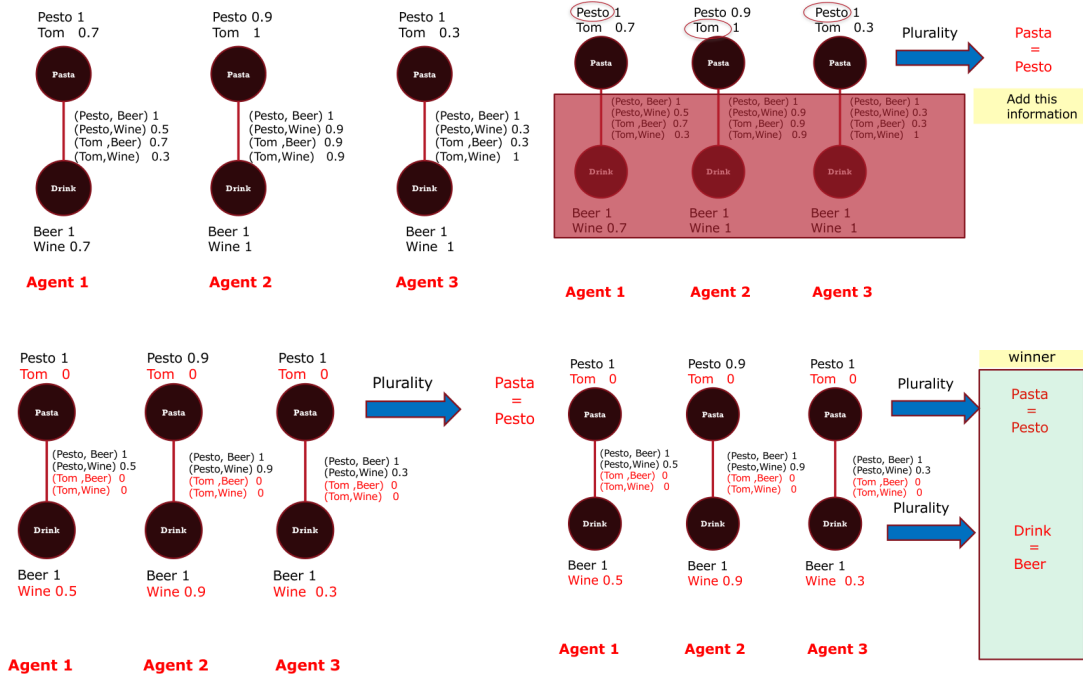
Figure 5.3: Soft constraints vs CP-nets.

make their vote for one issue depending on the decisions on previous issues.

In AI, this can be done via soft constraints or CP-nets. In sequential voting with soft constraint problem we assume to have the same constraint graph but different preference values. Then, for each variable:

1. Compute an explicit profile over the variable domain.
2. Apply a voting rule to this explicit profile: the rule will return a specific value that will be assigned to this variable.
3. Add the information about the selected variable value.

Similar approach used for CP-nets.



Chapter 6

Bayesian Networks

6.1 Introduction

Bayesian networks are **network models** that provide a framework to reason under uncertainty according to the laws of probability theory. They are a **graphical notation** used to represent dependencies among random variables and to specify a full joint probability distribution in a compact manner.

6.2 Syntax of Bayesian Networks

A Bayesian network consists of:

- A **directed acyclic graph (DAG)**.
- A set of **nodes**, one for each variable.
- A set of **directed edges (arcs)**, where an edge $X \rightarrow Y$ means that X directly influences Y .
- For each node X_i , a **conditional probability distribution (CPD)** given its parents:

$$P(X_i \mid \text{Parents}(X_i))$$

This is usually represented as a **conditional probability table (CPT)**.

6.2.1 Example: The Burglary Alarm Network

Consider the following scenario:

- A burglar alarm is installed at home. It is reliable at detecting burglaries but may also respond to minor earthquakes.
- Two neighbors, John and Mary, may call when they hear the alarm.
 - John always calls if he hears the alarm, but sometimes calls mistakenly (confuses the phone with the alarm).
 - Mary often misses the alarm because she listens to loud music.
- The task: Given evidence of who calls, estimate the probability of a burglary.

The variables are:

Burglary, Earthquake, Alarm, JohnCalls, MaryCalls

The causal structure is:

- Burglary $\rightarrow$ Alarm
- Earthquake $\rightarrow$ Alarm
- Alarm $\rightarrow$ JohnCalls
- Alarm $\rightarrow$ MaryCalls

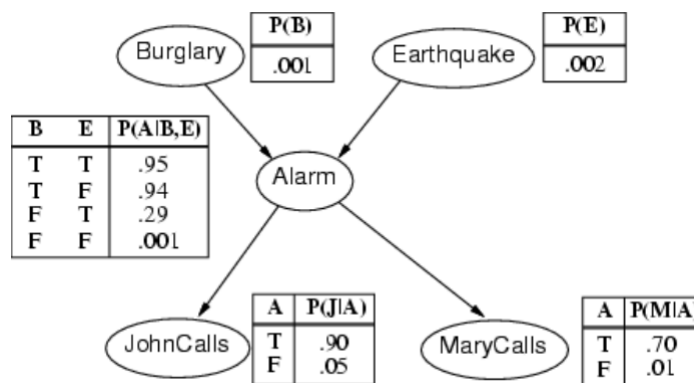


Figure 6.1

This structure reflects assumptions:

- John and Mary do not perceive burglaries or earthquakes directly.
- They act only based on the alarm.
- Other hidden factors (e.g. Mary's music, John confusing the phone) are summarized in the probabilities of the CPTs.

6.2.2 Conditional Probability Tables

Each node has a CPT. For Boolean variables:

- Each row gives $P(X_i = \text{true} \mid \text{Parents})$.
- Since $P(X_i = \text{false} \mid \text{Parents}) = 1 - p$, only one number is required per row.

Example:

$$P(B = \text{true}) = 0.001, \quad P(B = \text{false}) = 0.999$$

6.3 Compactness of Representation

6.3.1 Comparison with Full Joint Distribution

- A Bayesian network provides a much more compact representation than a full joint probability distribution.
- For n Boolean variables with at most k parents:

Bayesian network: at most $n \cdot 2^k$ numbers.

Full joint distribution: 2^n numbers.

For burglary network:

- Network requires only 10 numbers.
- Full joint distribution would require $2^5 - 1 = 31$ numbers.

6.4 Semantics of Bayesian Networks

The joint distribution over all variables is obtained by multiplying local conditional probabilities:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid \text{Parents}(X_i))$$

6.4.1 Example Calculation

For the burglary network:

$$P(j \wedge m \wedge a \wedge \neg b \wedge \neg e) = P(j \mid a) P(m \mid a) P(a \mid \neg b, \neg e) P(\neg b) P(\neg e)$$

Using given probabilities:

$$= 0.90 \times 0.70 \times 0.001 \times 0.999 \times 0.998 = 0.000628$$

6.5 Summary

Bayesian networks:

- Are directed acyclic graphs encoding conditional dependencies.
- Represent joint distributions compactly using local CPTs.
- Allow efficient reasoning about uncertainty.